Classification of order-ten binary 3-frameproof codes

A computer-assisted proof formalized in Lean

GPT-6 Astra

8 October 2026

Abstract

Every binary SHF$(10; 10, 2, \{1, 3\})$ is a permutation matrix up to independent complementation of rows. This resolves the order-ten case explicitly left open by Guo, Stinson and van Trung in 2015. The proof reduces a hypothetical nontrivial example to one of four canonical Boolean formulas on the entries of a 10×10 matrix. The reduction uses an elementary structural lemma, row complementation, and minimization over a finite permutation orbit. Each formula has 840 separation conditions, 80 row-order conditions, 63 column-order conditions, and 19 fixed-entry conditions. Lean verifies the analytic reduction, the semantic Boolean bridges, all four finite refutations, and their integration into the classification theorem. The finite refutations replay saved LRAT certificates through the standard `bv_check` procedure; their trust boundary includes Lean's compiled evaluation of the verified checker. We give the exact formulas, the mathematical proofs behind their normalization, and instructions for reproducing the formal result.

1 The question and its precise scope

Write $I = \{0, 1, \ldots, 9\}$ and identify the Boolean values false and true with 0 and 1. A binary matrix is a function $A : I \times I \to \{0, 1\}$. Rows are coordinates, or hash functions, and columns are codewords, or elements of the hash-family domain.

Definition 1.1. A row r *separates* a column a from columns b, c, d if

$$A_{ra} \neq A_{rb}, \qquad A_{ra} \neq A_{rc}, \qquad A_{ra} \neq A_{rd}. \tag{1}$$

The matrix A has the *separation property* if, for every four pairwise distinct columns $a, b, c, d \in I$, some row separates a from b, c, d.

This is the matrix definition of SHF$(10; 10, 2, \{1, 3\})$ used in [1]. In a binary row, the three entries on the right of (1) necessarily agree, although the definition does not need to state their agreement separately. Equivalently, no column can be reproduced coordinate by coordinate by choosing at each coordinate an entry from any fixed three other columns. The version of the frameproof definition allowing coalitions of size at most three is equivalent here: a smaller coalition can be extended to three other columns, and its coordinatewise descendants are contained in those of the larger coalition. There are enough columns to make this extension. Distinctness of columns is also forced by separation, since any two equal columns would prevent the required separation of one of them from a coalition containing the other.

Definition 1.2. The matrix A is *trivial* if there are a permutation p of I and bits $\epsilon_r \in \{0, 1\}$, one for each row, such that

$$A_{rc} = \mathbf{1}_{\{p(r)=c\}} \oplus \epsilon_r \qquad (r, c \in I), \tag{2}$$

where $\oplus$ denotes exclusive or.

Thus trivial means a permutation matrix up to independent row complementation. Additional row and column permutations are already absorbed in p. Conversely, every trivial matrix has the separation property: for designated column a, choose the unique row r with $p(r) = a$. Its entry in column a differs from every other entry in that row, and complementation preserves inequalities.

Theorem 1.3 (Classification)**.** *Every binary* 10×10 *matrix with the separation property is trivial. Equivalently, no nontrivial* $\mathrm{SHF}(10; 10, 2, \{1, 3\})$ *exists.*

The conclusion is an equality classification, not merely a bound on the number of codewords. The 2015 bound of Guo, Stinson and van Trung gives the corresponding uniqueness statement for the relevant orders $4 \le v \le 9$ [2]. Their paper on symmetric designs exhibits a nontrivial order-eleven example from a symmetric $(11, 5, 2)$ design and explicitly leaves order ten open in its concluding problem 3 [1]. Neither the problem nor the classification assumes that the matrix comes from a design, has constant row or column weight, or has a prescribed automorphism group.

The later binary wide-sense frameproof-code results of Zhao and Zhang [3] do not by themselves settle this equality case. Their quoted range $n \le 3t$ reaches nine when $t = 3$, while the range $n < ((15 + \sqrt{33})/24)(t - 1)^2$ does not reach ten. This is an applicability check on those results, not a claim that they establish the continued openness of the exact question. A literature audit completed on 8 October 2026 did not locate an earlier resolution. Its coverage is incomplete, including some inaccessible full texts; it cannot guarantee priority or exclude an unpublished result. In particular, the full text of [4] was not available in that audit.

2 An elementary structural lemma

Definition 2.1. A row r has a *private bit at column* a if $A_{rb} \neq A_{ra}$ for every $b \neq a$. A row is *elementary* if it is constant or has a private bit.

The binary alphabet is essential in the following argument: all entries different from a given bit are equal to one another.

Lemma 2.2. *A row has a private bit at at most one column.*

Proof. Suppose a row has private bits at distinct columns a and b. Choose $c \notin \{a, b\}$, which is possible because there are ten columns. Privacy at a implies that both A_{rb} and A_{rc} are the unique binary value different from A_{ra}, so $A_{rb} = A_{rc}$. This contradicts privacy at b. □

Lemma 2.3. *Suppose* $b \neq c$ *and an elementary row* r *satisfies* $A_{ra} \neq A_{rb}$ *and* $A_{ra} \neq A_{rc}$*. Then* r *has its private bit at* a*.*

Proof. The row cannot be constant. Let e be a column at which its bit is private. If $e = a$, the conclusion holds. Otherwise all entries outside e, including the entry at a, have the common bit different from A_{re}. The inequality at b forces $b = e$, and the inequality at c forces $c = e$. This contradicts $b \neq c$. □

Theorem 2.4 (Elementary-row classification)**.** *If* A *has the separation property and every row of* A *is elementary, then* A *is trivial.*

Proof. For each column a, choose three distinct other columns b, c, d. Separation supplies a row $f(a)$ separating a from those columns. Lemma 2.3, applied to b, c, shows that $f(a)$ has a private bit at a.

The map $f : I \to I$ is injective: if $f(a) = f(a')$, that row has private bits at a and a', so Lemma 2.2 gives $a = a'$. An injection between these two finite sets of size ten is a bijection.

Put $p = f^{-1}$. Every row r therefore has a private bit at column $p(r)$; in particular, the argument has also ruled out unused constant rows.

Define $\epsilon_r = 1 \oplus A_{r,p(r)}$. At $c = p(r)$, $1 \oplus \epsilon_r = A_{r,p(r)}$. At every $c \neq p(r)$, privacy and the binary alphabet give $A_{rc} = 1 \oplus A_{r,p(r)} = \epsilon_r$. These two cases are precisely (2). □

Corollary 2.5. *A nontrivial matrix with the separation property has a row containing at least two zeros and at least two ones.*

Proof. By the contrapositive of Theorem 2.4, some row is neither constant nor private at any column. Both bits occur because it is nonconstant. Neither occurs exactly once, because that occurrence would be a private bit. Thus both occur at least twice. □

No bound from the earlier classification through order nine is used in this structural proof.

3 Normalization and anchoring

Lemma 3.1 (Invariance of separation)**.** *The separation property is preserved by independent complementation of rows and by arbitrary permutations of rows and columns.*

Proof. For bits x, y, e, one has $x \oplus e \neq y \oplus e$ if and only if $x \neq y$. Consequently, complementing a row preserves each inequality in (1). A row permutation moves a witness to its inverse image. A column permutation sends a quartet of distinct columns to a quartet of distinct columns, to which the original separation property applies. Applying the inverse row permutation to the resulting witness proves the assertion. □

For a row r, put $C_r = \{c : A_{rc} = 1\}$ and $\mathrm{wt}(A_r) = |C_r|$. Complementing the row replaces C_r by its complement in I. For definiteness, use the following orientation: retain C_r if $|C_r| < 5$, or if $|C_r| = 5$ and $9 \notin C_r$; otherwise replace C_r by its complement. Denote the oriented set by $\widehat{C}_r$. Exactly one member of a complementary pair satisfies this rule, including when both members have size five. In particular, $|\widehat{C}_r| \leq 5$ for every row.

Lemma 3.2 (A distinguished first row)**.** *If A is a nontrivial matrix with the separation property, then after row complements and row and column permutations it has a representative N and an integer $k \in \{2, 3, 4, 5\}$ such that*

$$N_{0c} = \mathbf{1}_{\{c<k\}} \qquad (c \in I). \tag{3}$$

One may additionally arrange that every row of N has weight at most five and has weight at most one or at least k.

Proof. Orient every row as above. By Corollary 2.5, at least one oriented row has weight between two and five: complementation preserves the assertion that both bits occur at least twice. Choose an oriented row of smallest weight among those of weight at least two, and call its weight k. This gives $2 \leq k \leq 5$ and the claimed restrictions on the weights of all other rows.

Move the chosen row to index zero. Let C be its set of ones, so $|C| = k$. Choose any bijection from $\{0, \ldots, k-1\}$ to C, and any bijection from $\{k, \ldots, 9\}$ to $I \setminus C$. Their union is a permutation p of I. Replacing column c by column $p(c)$ gives (3). Column permutations preserve row weights, and Lemma 3.1 preserves separation throughout. □

Lemma 3.3 (Anchored representative)**.** *The representative in Lemma 3.2 can be changed by further row complements to a matrix B with the separation property, the same first row* (3)*, and*

$$B_{r0} = 0 \qquad (1 \leq r \leq 9). \tag{4}$$

Proof. Leave row zero unchanged. For $r > 0$, replace row r by $B_{rc} = N_{rc} \oplus N_{r0}$. Its entry at column zero is $N_{r0} \oplus N_{r0} = 0$. Separation is preserved by Lemma 3.1, and the first row is unchanged. □

Remark 3.4. After this anchoring step, no bound on the weights of rows $1, \ldots, 9$ is retained or used. Complementing a light row can make it heavy. The only weight specification needed subsequently is the explicit first row in (3). In particular, the finite formulas below contain no additional row-weight or minimum-weight conditions.

4 A simultaneous lexicographic representative

Use the order $0 < 1$. For two finite binary sequences u, v of the same length, write $u \leq_{\text{lex}} v$ for the usual lexicographic order, comparing coordinates in increasing index order. Equivalently,

$$\text{for each index } c, \quad \big(\forall t < c,\ u_t = v_t\big) \implies u_c \leq v_c. \tag{5}$$

Indeed, if the sequences are distinct, the condition at their first unequal coordinate says that its pair of bits is $(0, 1)$. Conversely, the first unequal coordinate of a lexicographically ordered pair prevents a later coordinate from having an equal prefix.

Fix $k \in \{2, 3, 4, 5\}$. The residual row permutations fix row zero. The residual column permutations fix column zero and permute separately the two sets

$$U_k = \{1, \ldots, k-1\}, \qquad V_k = \{k, \ldots, 9\}. \tag{6}$$

Both sets are nonempty. These permutations preserve (3) and (4).

Lemma 4.1 (Finite-orbit double lexicographic ordering)**.** *An anchored matrix B satisfying* (3) *has a representative D under the residual permutations such that, writing $D_c^- = (D_{1c}, \ldots, D_{9c})$ for a truncated column,*

$$D_i \leq_{\text{lex}} D_j \qquad (1 \leq i < j \leq 9), \tag{7}$$

$$D_c^- \leq_{\text{lex}} D_d^- \qquad (c < d,\ c, d \in U_k \text{ or } c, d \in V_k). \tag{8}$$

The first row and the anchor remain unchanged.

Proof. The orbit under the residual permutations is finite and nonempty. Flatten each matrix in row-major order,

$$(D_{00}, D_{01}, \ldots, D_{09}, D_{10}, \ldots, D_{99}),$$

and choose a lexicographically least flattened matrix D in the orbit.

If (7) failed for $i < j$, then $D_j <_{\text{lex}} D_i$. Swap rows i and j, an allowed residual permutation. All rows before i remain unchanged. Within row i, the first difference from the original matrix changes a 1 to a 0, and every preceding entry is unchanged. The new flattened matrix is smaller, a contradiction.

Suppose (8) failed for a permitted pair $c < d$. Let $r > 0$ be the first row at which these truncated columns differ. Failure means $D_{rc} = 1$ and $D_{rd} = 0$, and $D_{sc} = D_{sd}$ for $1 \leq s < r$. Also $D_{0c} = D_{0d}$ because the two columns belong to the same first-row block. Swap columns c, d, which is an allowed permutation. Every entry in rows before r is unchanged: the swapped entries were equal. In row r, entries before column c are unchanged, while the entry at c changes from 1 to 0. This again strictly decreases the flattened matrix, a contradiction. Finally, all orbit elements satisfy the fixed first row and zero anchor by the definition of the residual group. □

This argument establishes simultaneous ordering in one finite orbit; it does not assume that separately sorting rows and columns leaves the other ordering intact.

Theorem 4.2 (Canonical reduction)**.** *If a nontrivial matrix with the separation property exists, then for some $k \in \{2,3,4,5\}$ there exists a matrix D with the separation property satisfying* (3)*,* (4)*,* (7)*, and* (8)*.*

Proof. Apply Lemma 3.2, then Lemma 3.3, and then Lemma 4.1. Lemma 3.1 preserves the separation property at each step. The last step fixes the first row and anchor. There are no other conditions on D. □

5 The four explicit Boolean formulas

For each fixed $k \in \{2,3,4,5\}$, introduce the 100 Boolean variables x_{rc} with $r, c \in I$. Every empty conjunction below has value true. An equality between Boolean expressions is interpreted as its Boolean equality test when it occurs inside a formula.

5.1 Separation: 840 conditions

For $a \in I$ and $b < c < d$ in $I \setminus \{a\}$ define

$$S_{a;bcd}(x) = \bigvee_{r=0}^{9} \big((x_{ra} \oplus x_{rb}) \wedge (x_{ra} \oplus x_{rc}) \wedge (x_{ra} \oplus x_{rd})\big). \tag{9}$$

There are $10\binom{9}{3} = 840$ such conditions, each required to be true. Ordering b, c, d removes duplicates only: separation is symmetric in these three opposing columns. The designated column a is not interchanged with them or omitted from the indexing.

5.2 The first row and the anchor: 19 conditions

Require the ten equations and nine equations

$$x_{0c} = \mathbf{1}_{\{c<k\}} \quad (0 \le c \le 9), \qquad x_{r0} = 0 \quad (1 \le r \le 9). \tag{10}$$

In particular $x_{00} = 1$, so the anchor does not include row zero.

5.3 Adjacent row ordering: 80 conditions

For $1 \le i \le 8$ and $0 \le c \le 9$, put

$$\begin{aligned} P_{i,c}(x) &= \bigwedge_{0 \le t < c} (x_{it} = x_{i+1,t}), \\ R_{i,c}(x) &= \neg P_{i,c}(x) \ \vee \ \neg x_{ic} \ \vee \ x_{i+1,c}. \end{aligned} \tag{11}$$

Require all $8 \cdot 10 = 80$ formulas $R_{i,c}$ to be true. By (5), these say that each adjacent pair of free rows is lexicographically ordered. Transitivity then orders all free rows. The stronger all-pairs conclusion of Lemma 4.1 directly supplies every required adjacent condition.

5.4 Adjacent residual column ordering: 63 conditions

The permitted left endpoints of adjacent column pairs are

$$J_k = \{1, \dots, k-2\} \ \cup \ \{k, \dots, 8\}, \tag{12}$$

where the first set is empty for $k = 2$. These are precisely the pairs $(c, c+1)$ lying inside U_k or V_k. They neither compare column zero with another column nor compare across the block boundary. For $c \in J_k$ and $1 \le r \le 9$ define

$$\begin{aligned} Q_{c,r}(x) &= \bigwedge_{1 \le s < r} (x_{sc} = x_{s,c+1}), \\ L_{c,r}(x) &= \neg Q_{c,r}(x) \ \vee \ \neg x_{rc} \ \vee \ x_{r,c+1}. \end{aligned} \tag{13}$$

Require every $L_{c,r}$ to be true. Since $|J_k| = (k-2) + (9-k) = 7$, there are exactly $7 \cdot 9 = 63$ conditions in every case. These compare the truncated columns with row zero omitted, in agreement with (8).

5.5 The combined formula and its exact meaning

Let $F_k(x)$ be the conjunction of the 19 equations in (10). Define

$$\begin{aligned} \Phi_k(x) = F_k(x) \ \wedge \bigwedge_{\substack{a \in I \\ b<c<d, \ b,c,d \neq a}} S_{a;bcd}(x) \\ \wedge \bigwedge_{\substack{1 \le i \le 8 \\ 0 \le c \le 9}} R_{i,c}(x) \ \wedge \bigwedge_{\substack{c \in J_k \\ 1 \le r \le 9}} L_{c,r}(x). \end{aligned} \tag{14}$$

There are $840 + 80 + 63 + 19 = 1002$ high-level conjuncts. This is a count before constant simplification. Some become tautologies after the fixed entries are substituted. It is not a count of clauses in a particular auxiliary-variable CNF encoding: for example, (9) contains conjunctions inside a disjunction.

Proposition 5.1 (Exact reduction of existence)**.** *A nontrivial binary* SHF$(10; 10, 2, \{1, 3\})$ *exists if and only if at least one of* $\Phi_2, \Phi_3, \Phi_4, \Phi_5$ *is satisfiable.*

Proof. For the forward implication, apply Theorem 4.2 and assign $x_{rc} = D_{rc}$. Its separating witnesses make every $S_{a;bcd}$ true. The first row and anchor give F_k, and the lexicographic prefix implications give $R_{i,c}$ and $L_{c,r}$. Thus Φ_k is true.

Conversely, interpret a satisfying assignment as a binary matrix X. For any designated column a and any three distinct other columns, sort those three indices as $b < c < d$. The corresponding $S_{a;bcd}$ gives a row separating a from all three, independently of their order. Therefore X has the separation property. Its first row has exactly k ones by F_k. In a trivial matrix every row has weight either one or nine, as follows immediately from (2). Since $2 \le k \le 5$, X is nontrivial. □

In particular, the symmetry conditions have not silently removed every representative of a possible counterexample. The forward implication is established by a constructive sequence of transformations followed by a minimum over a finite orbit.

6 The finite refutations and the classification theorem

Lemma 6.1 (Finite Boolean refutations)**.** *For each $k \in \{2,3,4,5\}$, the formula Φ_k is unsatisfiable:*

$$\forall x \in \{0,1\}^{I\times I}, \qquad \neg\Phi_k(x). \tag{15}$$

Computer-assisted proof. The Lean development proves the semantic interpretation of every conjunct in (14). It expands the resulting finite Boolean expression and canonicalizes the concrete matrix indices using checked definitional equality. The four expressions are refuted by replaying the saved certificates `certificates/case2.lrat`, `certificates/case3.lrat`, `certificates/case4.lrat`, and `certificates/case5.lrat` with Lean's standard `bv_check` procedure. Each replay proves the corresponding Boolean contradiction; no solver answer is inserted as an unproved mathematical hypothesis. Appendix B specifies the expression construction, normalization, and certificate method. Section 7 states its compiled-evaluation trust boundary and reproduction commands. □

Proof of Theorem 1.3. Suppose A has the separation property but is not trivial. Proposition 5.1 supplies $k \in \{2,3,4,5\}$ and a satisfying assignment of Φ_k. This contradicts Lemma 6.1 for that value of k. Hence A is trivial. □

6.1 Exact interface used by the Lean case proofs

The development uses a direct matrix interface. For each $k \in \{2,3,4,5\}$, the theorem `anchored_case_k` takes the following arguments: a binary matrix A, its separation property, its fixed first row, its zero anchor, the all-pairs ordering of free rows, and the ordering of residual columns. Its conclusion is false. Each such theorem obtains the 1002 concrete Boolean facts through proved semantic lemmas and then refutes their conjunction.

The Boolean row-prefix implementation is the explicit finite formula

$$\bigwedge_{t=0}^{9}\big((c \le t) \vee (x_t = y_t)\big).$$

For $t \ge c$ the first disjunct is true; for $t < c$ it reduces to $x_t = y_t$. A checked lemma identifies this formula with equality on all earlier coordinates. The column-prefix version uses $t = 1,\dots,9$ and likewise omits row zero. The Boolean separation implementation expands the existential row witness into exactly ten disjuncts. Thus these helper definitions have the meanings of (9), (11), and (13), not merely similar-looking encodings.

The formal canonicalization theorem produces an integer k satisfying $2 \le k \le 5$ together with precisely the five matrix properties used by the case theorems. Case analysis on this interval gives the four alternatives. The final integration proves the unconditional statements

```
every_order_ten_shf_is_trivial
  (A : BinaryMatrix 10) (h : IsSHF A) : IsTrivial A

not_openProblem10 : Not OpenProblem10
```

Here `OpenProblem10` is the definition $\exists A,\ \mathrm{IsSHF}(A) \wedge \neg\mathrm{IsTrivial}(A)$; it is not an axiom or a hypothesis of the final classification.

7 Formal verification and trust boundary

The project uses the following pinned Lean and mathlib sources [5, 6]. The displayed identifiers also appear in the project metadata and Lake dependency manifest.

```
Lean version: 4.34.1
Lean commit:
5045d0056413266e57c625dcd7c365b10e377c52
mathlib commit:
d13f23b723b8a846827a245b89c10fc7d3f11612
```

The completed statements are `GPT6Astra.every_order_ten_shf_is_trivial` and `GPT6Astra.not_openProblem10` in `MainTheorem.lean`. The structural, normalization, finite-orbit, and semantic bridge proofs use the standard foundational axioms `propext`, `Classical.choice`, and `Quot.sound`. The final classification additionally depends on the evaluation axioms generated by the standard `bv_check` calls for the four finite cases. The actual dependency lists are printed by `#printaxioms` in the case modules and in `MainTheorem.lean`, and retained in the verification logs. No proof is admitted using `sorry`, and no user-written axiom asserts a combinatorial or computational conclusion.

The additional evaluation dependency is substantive. Lean's normalization and reflection code connects the matrix expressions to Boolean and bit-vector expressions and then to the certificate problem [5]. Its standard library proves soundness of the LRAT checker [7]. In the standard `bv_check` path used here, the concrete checker calculation is performed by compiled evaluation and is imported into the proof through generated evaluation axioms. Consequently, reproduction trusts that evaluation path, including the compiler and runtime that execute the checker, in addition to the usual Lean foundations and kernel. The proof is not claimed to reconstruct every certificate step solely through kernel reduction. The external SAT search program is not trusted to decide truth: a saved certificate must pass the checker, and replay does not require another solver search.

For corroboration, a separately generated Python CNF encoding of each of the four anchored cases was also refuted by certificate-derived Lean proof terms whose reported axiom dependency was only `propext`. Those independent audits establish unsatisfiability of the precise CNFs they inspect. They supplement the final matrix proof; they neither substitute for its semantic-to-Boolean-to-certificate chain nor remove its stated compiled-evaluation dependency.

7.1 Reproduction and immutable proof data

The stable replay files are `certificates/caseK.lrat` for $K = 2, 3, 4, 5$, relative to the Lean project directory. The distribution's `SHA256SUMS` records the actual hashes of the final sources and certificates. Reproduction should use those recorded source and certificate bytes. The checked source modules use `bv_check` with these stable paths. The generator in `research/direct-matrix/generate_compact_cases.py` records the indexing scheme and can reproduce the source layout, but it does not supply an axiom or bypass Lean verification.

With the pinned toolchain and dependencies installed, enter the `gpt6_astra` project directory. Its `./lake` wrapper selects the project toolchain. The following commands replay the cases sequentially, limiting peak memory use, and then build the integrated theorem:

```
sha256sum -c SHA256SUMS
./lake build FiniteCase2
./lake build FiniteCase3
./lake build FiniteCase4
./lake build FiniteCase5
./lake build MainTheorem
```

Running `./lakebuildMainTheorem` after these case builds reuses the checked dependencies. A successful replay means the commands terminate successfully and the final printed dependencies contain no `sorryAx`; the generated compiled-check evaluation axioms are expected

and explicitly disclosed above. The proof certificate logs and hash manifest identify the exact release being reproduced.

A Correspondence with the Lean development

All listed declarations are in namespace `GPT6Astra`. The table maps the mathematical proof and computational interface to the checked source files.

Mathematical statement	**File and declarations**
Definitions 1.1 and 1.2	`CryptographyResearch.lean`: `BinaryMatrix`, `Separates`, `IsSHF`, `IsTrivial`, `OpenProblem10`.
Row-complement invariance	`CryptographyResearch.lean`: `separates_complementRows_iff`, `isSHF_complementRows_iff`.
Lemmas 2.2 and 2.3	`MatrixStructure.lean`: `PrivateAt`, `privateAt_unique`, `privateAt_of_elementary_separatesPair`.
Theorem 2.4 and Corollary 2.5	`MatrixStructure.lean`: `isTrivial_of_all_rows_elementary`, `exists_non_elementary_row`.
Oriented row sets and a nontrivial row	`CutFamilies.lean`: `normalizeCut`, `normalizeCut_canonical`, `selectedCuts_has_minimal_nontrivial`. The selected cuts are deduplicated sets used to choose a witness; this does not assume distinct rows in the original matrix.
Lemma 3.2	`MatrixNormalization.lean`: `normalizedMatrix_isSHF`, `permuteMatrix_isSHF`, `exists_perm_to_initialCut`, `exists_normalized_matrix`.
Lemma 3.3	`MatrixNormalization.lean`: `AnchorZero`, `anchoredMatrix_isSHF`, `anchoredMatrix_fixedRow`, `anchoredMatrix_anchorZero`, `exists_anchor_normalized_matrix`.
Lemma 4.1	`DoubleLex.lean`: `matrixLexKey`, `rowSwap_key_lt`, `columnSwap_key_lt`, `exists_doubleLex_anchored`, `exists_doubleLex_anchored_fixed`.
Theorem 4.2	`CanonicalMatrix.lean`: `exists_anchored_canonical_matrix`. The conclusion omits `Shape`; see Remark 3.4.
The 840 separation conditions	`FiniteCaseSupport.lean`: `separatesB`, `separationClause`, `separationClause_of_isSHF`.
The Boolean lexicographic conditions	`LexBoolean.lean`: `prefixB_eq_true_iff`, `rowLexClause_of_doubleLexRows`, `columnPrefixB_eq_true_iff`, `columnLexClause_of_doubleLexAnchoredColumns`.
The four finite-case refutations	`FiniteCase2.lean` through `FiniteCase5.lean`; statements `anchored_case_2` through `anchored_case_5`.
The unconditional classification	`MainTheorem.lean`; `every_order_ten_shf_is_trivial` and `not_openProblem10`.

The paper also proves the converse direction in Proposition 5.1 to make the meaning of the formulas transparent. The formal classification only needs its forward direction: a nontrivial separating matrix would satisfy the finite constraints. Auxiliary pair-separation lemmas and alternative cut-family SAT encodings are not assumptions of the anchored four-case argument presented here.

B Exact computational proof method

The formal computation operates on the matrix expressions directly; the separately generated Python CNFs are not inputs to the final classification proof. The following steps describe the complete computational part of each case module.

Index generation. For fixed k, the source generator emits the first-row conditions in column order $0, \ldots, 9$ and the anchor conditions in row order $1, \ldots, 9$. It next emits row-prefix conditions for $i = 1, \ldots, 8$ and $c = 0, \ldots, 9$. Column-prefix conditions are generated from adjacent pairs within the actual lists $[1, \ldots, k-1]$ and $[k, \ldots, 9]$, with prefix rows $1, \ldots, 9$. Finally it enumerates each designated column $a = 0, \ldots, 9$ and each increasing triple from $I \setminus \{a\}$. Thus all case-specific indices are recomputed; they are not obtained by replacing numerals inside another case's formula. The same order is retained when replaying a certificate.

Semantic proof blocks. The 1002 conditions are grouped into 31 blocks of 32 conditions and a last block of ten. Each block is a balanced Boolean conjunction. Every leaf has a proof obtained from a fixed-entry assumption or from one of the semantic lemmas listed in Appendix A. The local helper `bool_and_intro` combines proofs of $a = \text{true}$ and $b = \text{true}$ into a proof of $(a \mathbin{\&\&} b) = \text{true}$. The 32 block proofs are then combined by another balanced conjunction. This layout avoids a local context with more than a thousand independent hypotheses without changing the conjunction being proved.

Expansion and canonical atoms. The definitions of the combined constraint, its blocks, and the leaf Boolean operations are unfolded. The tactic `canonical_matrix_atoms A at hc` visits the resulting expression. For a concrete matrix application $A\,r\,c$, it computes the values of its two finite indices and replaces the application with the cached canonical expression for that pair. It also reduces closed Boolean `decide` expressions, including the numerical comparisons in fixed-row and prefix conditions. This step is needed because distinct syntactic representations of the same finite index can otherwise be abstracted as different Boolean atoms. It does not add an equality as an assumption: Lean's `changeLocalDecl` is called with its definitional-equality check enabled. Acceptance therefore confirms that the rewritten hypothesis has the same meaning as the original.

Certificate replay. The replay call uses the standard Boolean decision procedure with the following configuration, where K is the fixed case number:

```
bv_check
  (config := { timeout := 120, maxSteps := 2000000,
    structures := false, enums := false, fixedInt := false,
    embeddedConstraintSubst := false })
  "certificates/caseK.lrat"
```

The generated source writes the configuration on one line and escapes the field name `enums` as required by its Lean syntax; the multiline display above is explanatory. The configuration retains the same normalization used for certificate generation, while disabling passes for structures, enumerations, fixed-width machine integers, and embedded-constraint substitution that are unnecessary for this explicit Boolean problem. The step budget controls simplification; it is not a mathematical restriction on possible matrices. The `timeout` field is inherited from the search configuration and does not turn an incomplete check into a proof.

The resulting theorem `anchored_case_K` concludes false from the five semantic properties of its input matrix. Combining the four checked case theorems with `exists_anchored_canonical_matrix`, and splitting the integer interval $2 \le k \le 5$, proves `every_order_ten_shf_is_trivial`. Unpacking the existential `OpenProblem10` and applying that theorem proves `not_openProblem10`. There is no additional hypothesis concerning the solver, the certificate, regularity, or symmetry in either final mathematical statement.

References

[1] C. Guo, D. R. Stinson, and T. van Trung, "On Symmetric Designs and Binary 3-Frameproof Codes," in C. J. Colbourn, ed., *Algebraic Design Theory and Hadamard Matrices*, Springer Proceedings in Mathematics & Statistics 133, Springer, 2015, pp. 125–136. doi:10.1007/978-3-319-17729-8_10. Author manuscript: `https://cs.uwaterloo.ca/~dstinson/papers/sdesign-5.pdf`.

[2] C. Guo, D. R. Stinson, and T. van Trung, "On tight bounds for binary frameproof codes," *Designs, Codes and Cryptography* **77**(2–3) (2015), 301–319. doi:10.1007/s10623-015-0037-y. Preprint: `https://arxiv.org/abs/1406.6920`.

[3] Y. Zhao and X. Zhang, "Improved Upper Bounds for Wide-Sense Frameproof Codes," *IEEE Transactions on Information Theory* **70**(12) (2024), 8636–8646. doi:10.1109/TIT.2024.3411150. Version inspected in the literature audit: `https://arxiv.org/html/2402.05596v1`.

[4] C. Sun and X. Wang, "New upper bounds for wide-sense frameproof codes," *Designs, Codes and Cryptography* **93**(8) (2025), 3069–3082. doi:10.1007/s10623-025-01631-x. Only the abstract and first-page preview were available in the literature audit reported here.

[5] Lean FRO and the Lean contributors, *Lean 4*, version 4.34.1, official source distribution. Boolean decision-procedure frontend: `https://github.com/leanprover/lean4/blob/v4.34.1/src/Lean/Meta/Tactic/BVDecide/Main.lean`.

[6] The mathlib community, *mathlib4*, version 4.34.1, official release and source repository. `https://github.com/leanprover-community/mathlib4/releases/tag/v4.34.1`. The exact dependency commit used here is recorded in Section 7.

[7] J. Clune and the Lean contributors, *LRAT checker soundness*, Lean standard-library source, version 4.34.1. `https://github.com/leanprover/lean4/blob/v4.34.1/src/Std/Tactic/BVDecide/LRAT/Internal/LRATCheckerSound.lean`.